

Énumération et génération aléatoire avec python

1 Rappels sur python nécessaire pour ce TP

1.1 Langage non typé

On rappelle que **le langage python est un langage non typé**. Ainsi lorsque l'on met un argument à une fonction de python, la valeur associée à cet argument lors d'un appel de la fonction peut être de plusieurs types pourvu que ceux-ci définissent toutes les instructions comportant cet argument qui sont utilisées dans la fonction. Nous devons bien avoir en tête ce principe à chaque fois que nous définissons une fonction et à chaque fois que nous souhaitons utiliser une fonction.

Un langage non typé apporte plus de souplesse dans l'écriture des programmes, mais il amène également plus d'erreurs de typage car celles-ci ne sont pas détectées.

Exemple

```
def foisDeux(x):  
    return 2*x  
a = 10  
b = "Bonjour"#type str (chaîne de caractères)  
print(foisDeux(a))#on affiche 20  
print(foisDeux(b))#on affiche BonjourBonjour
```

- x peut être de n'importe quel type
- il faut juste que l'opération $2 * x$ soit définie
- a étant un int, $2 * a$ est la multiplication sur les entiers
- b étant une chaîne de caractères, $2 * b$ est la duplication de b

1.2 Lecture-écriture

```
x = input("Entrez un entier")#par défaut x est de type str  
y = foisDeux(x)  
print(y)#si x vaut 7 on affiche 77  
#pour lire un entier  
x = int(input("Entrez un entier"))#conversion de l'entrée en entier  
y = foisDeux(x)  
print(y)
```

1.3 Si alors sinon

```
x = int(input("Entrez un entier"))  
if x%2==0:  
    print(x, " est un entier pair")  
else:  
    print(x, " est un entier impair")
```

L'instruction `else` est facultative. Pour imbriquer plusieurs `if`, on utilise l'instruction `elif`.

1.4 Boucles

Nous avons deux types de boucles, les boucles **while** et les boucles **for**.

Nous essaierons d'appliquer à chaque fois le principe suivant :

1) boucle **while** lorsque l'on ne sait pas combien de fois on entre dans la boucle :

```
x = int(input("Entrez un entier"))
while x%2==0:
    print(x)
    x = int(input("Entrez un entier"))
```

2) Boucle **for** lorsque l'on sait combien de fois on entre dans la boucle :

```
for i in range(10):
    print(i)
for i in range(45,-14,-4):
    print(i)
```

1.5 Fonction python

La fonction suivante retourne la somme des entiers de 1 à n .

```
def somme(n):
    res = 1
    for i in range(2,n+1):
        res += i
    return res
x = int(input("Entrez un entier"))
print(somme(x))
```

1.6 Listes et tuples

Les deux types permettent de stocker une suite de valeurs qui peuvent être de types différents. On peut utiliser des slices pour accéder à un élément ou une partie de la liste :

- `L[1]` renvoie l'élément en position 1 (le second élément de la liste, car on part de la position 0)
- `L[a:b]` renvoie une nouvelle liste avec les éléments entre les positions a et $b - 1$

Les listes sont itérables, nous pouvons utiliser l'instruction `for element in L` afin de parcourir tous les éléments de la liste (voir ci-dessous).

```
L = [5, 2, 8, 9]
for i in range(len(L)):
    print(L[i])
for element in L:
    print(element)
def premiereMoitieListe(L):
    return L[:len(L)/2]
L2 = premiereMoitieListe(L)
```

On définit un tuple en remplaçant les crochets par des parenthèses

```
pointD2 = (4,8)
pointD3 = (4,2,9)
```

Les éléments d'un tuple ne peuvent être modifiés, l'instruction `pointD2[0] = 5` est illicite. On dit qu'un tuple est un objet **non mutable**, contrairement à une liste qui est un objet **mutable**.

2 Énumération des éléments d'un ensemble

2.1 Itérateurs et générateurs python

Les **itérables** sont des objets qui contiennent plusieurs éléments et qui possèdent un **itérateur** qui permet d'accéder à ces éléments un par un. L'instruction pour énumérer les éléments d'un objet itérable est `for element in objetIterable`

```
def afficheIterable(objetIterable):
    for element in objetIterable:
        print(element,end=' ')
    print('')
```

Question 1 Testez cette fonction avec différents itérables comme des chaînes de caractères, les entiers naturels de a à b (`range(a,b+1)`), des listes, des tuples.

Solution

```
mot = "Belle journée"
decimaux = range(10)
point3d = (4,8,-12)
maListe = [9,'a',True]#une liste python peut contenir des éléments de tout type
afficheIterable(mot)
afficheIterable(decimaux)
afficheIterable(point3d)
afficheIterable(maListe)
```

L'itérateur sur un objet est défini dans la classe de cet objet. Cela nécessite donc quelques connaissances en POO (programmation orientée objet). Cependant nous avons à notre disposition des générateurs qui eux ne nécessitent aucune connaissance en POO et sont définis avec une fonction. L'instruction `yield` permet de renvoyer le prochain élément, elle est appelée soit avec l'instruction `for element in objetIterable` (faites attention à ne pas appliquer une boucle infinie lorsque le nombre d'éléments est infini) comme pour l'itérateur, soit avec l'instruction `next(générateur)` (testez s'il reste des éléments lorsque le nombre d'éléments est fini).

Exemple 1 Générateur des chiffres.

On lève une exception lorsqu'il n'y a plus d'élément à générer.

```
def generateurChiffres():
    c = 0;
    while (c < 10):
        yield c
        c +=1
gen = generateurChiffres()
encore = 1
while encore:
    try:
        print(next(gen))
    except:
        print("C'est fini")
        encore = 0
```

Exemple 2 Générateur énumérant tous les entiers naturels.

```
def generateurEntiersNaturels():
    i = 0
    while True:
        yield i
        i += 1
gen = generateurEntiersNaturels()
print(next(gen))
print(next(gen))
```

Question 2 Donnez un générateur qui énumère tous les nombres de $\mathbb{Z}$ avec l'ordre $0, 1, -1, 2, -2, \dots$

Solution

```
def generateurEntiersRelatifs():
    yield 0
    i = 1
    while True:
        yield i
        yield -i
        i += 1
gen = generateurEntiersRelatifs()
i = 0
for element in gen:
    print(element)
    i += 1
    if i > 20:
        break
```

2.2 Listes et générateurs en compréhension

Il est possible de construire une liste directement en réalisant une itération d'un objet itérable.

```
L = [_4,3,8,-2,7,-9,-1]
L1 = [x**2 for x in L]# L1 = [16, 9, 64, 4, 49, 81, 1]
L2 = [x for x in L if x>0]# L2 = [4, 3, 8, 7]
#on filtre les éléments strictement positifs de L
L3 = ["Pair" if x%2==0 else "Impair" for x in L]
#L3 = ['Pair', 'Impair', 'Pair', 'Pair', 'Impair', 'Impair', 'Impair']
```

Notez bien que le `if` se trouve avant la boucle `for` lorsqu'il est suivi d'un `else`, c'est-à-dire nous avons une valeur pour chaque élément itéré. En revanche, lorsqu'il s'agit de filtrer, le `if` se trouve après le `for`.

Pour obtenir un générateur, il suffit de remplacer les crochets `[,]` par des parenthèses `(,)`.

```
L = [4,3,8,-2,7,-9,-1]
gen1 = (x**2 for x in L)
gen2 = (x for x in L if x>0)
gen3 = ("Pair" if x%2==0 else "Impair" for x in L)
```

Soit L une liste, l'instruction `gen = (e for e in L)` retourne un générateur `gen` énumérant tous les éléments de L dans l'ordre $L[0], L[1], \dots$

De manière générale, si E est un objet itérable, alors `gen = (e for e in L)` retourne un générateur énumérant tous les éléments de E , mais l'ordre n'est pas toujours connu : voir par exemple les objets de la classe `set` (ensembles) et ceux de la classe `dict` (les dictionnaires), de plus l'ordre peut être différent d'une fois sur l'autre.

Il est possible d'imbriquer plusieurs boucles `for`.

```
L = [ (x,y) for x in range(3) for y in range(10)]
gen = ((x,y) for x in range(3) for y in range(10))
```

2.3 Produit cartésien

Définition 2.1 (*Produit cartésien*)

Soient A et B deux ensembles finis de cardinalités respectives n_1 et n_2 .

On définit le produit cartésien de A par B par

$$E = A \times B = \{(a, b) \mid a \in A \text{ et } b \in B\}.$$

Question 3 Définissez (une ligne de code avec une liste en compréhension) une fonction `listeProduitCartesien(A,B)` qui retourne une liste contenant tous les éléments de E .

Modifiez cette fonction en une fonction `generateurProduitCartesien(A,B)` qui retourne un générateur énumérant tous les éléments de E .

Solution

```
def listeProduitCartesien(A,B):
    return [ (a,b) for a in A for b in B]

def generateurProduitCartesien(A,B):
    return ( (a,b) for a in A for b in B)
```

Question 4 Utilisez la fonction `generateurProduitCartesien(A,B)` pour énumérer les cartes d'un jeu de 32 cartes. Vous ne définirez pas de nouvelle fonction.

Solution

```
hauteur = [7,8,9,10,'V','D','R','A']
couleur = ["trèfle","carreau","coeur","pique"]
jeu32 = generateurProduitCartesien(hauteur,couleur)
afficheIterable(jeu32)
```

2.4 Parties et combinaisons d'un ensemble fini

Définition 2.2 (*Ensemble des parties d'un ensemble fini*)

Soit E un ensemble fini. L'ensemble des parties de E est défini par

$$\mathcal{P}(E) = \{F \mid F \subset E\}.$$

Nous avons donc l'équivalence entre $F \in \mathcal{P}(E)$ et $F \subset E$.

Définition 2.3 (*Fonction caractéristique ou fonction indicatrice*)

Soit E un ensemble fini et $F \in \mathcal{P}(E)$. La fonction caractéristique χ_F est définie par

$$\begin{array}{lll} \chi_F : E & \rightarrow & \{0,1\} \\ e & \mapsto & 1 \quad \text{lorsque } e \in F \\ & & 0 \quad \text{lorsque } e \notin F \end{array}$$

Question 5 On utilise la fonction caractéristique afin de générer toutes les parties de E .

Vérifiez que la fonction `generateurParties(E)` suivante permet de générer tous les éléments de $\mathcal{P}(E)$.

Vous exécuterez cette fonction en prenant des ensembles E de petite cardinalité. On suppose ici que tous les ensembles sont définis par une liste.

```
def generateurParties(E):
    if len(E) == 0:
        yield []
    else:
        for partie in generateurParties(E[1:]):
            yield partie
            yield [E[0]]+partie
```

Le principe est le suivant : soit n la cardinalité de E , notons $E = \{e_1, \dots, e_n\}$ et $E_i = \{e_i, \dots, e_n\}$ pour tout $i \in \{1, \dots, n\}$. La construction des parties de E s'effectue de manière récursive. Soit $i \in \{1, \dots, n-1\}$, supposons que nous ayons un générateur des parties F de E_i , nous obtenons un générateur des parties de E_{i-1} en générant alternativement les parties F et $F \cup \{e_{i-1}\}$.

Définition 2.4 (*Combinaison*)

Soit E un ensemble de cardinal n . Une combinaison de p éléments de E parmi les n éléments de E est une partie de E de cardinal p .

L'ensemble des combinaisons de p éléments de E est donc

$$\mathcal{C}_p(E) = \{F \in \mathcal{P}(E) \mid \text{card}(F) = p\}.$$

Définition 2.5 (*Coefficient binomial*)

Le nombre de combinaisons de p éléments d'un ensemble à n éléments est appelé coefficient binomial et noté $\binom{n}{p}$ (ou encore C_n^p).

Les coefficients binomiaux s'expriment à l'aide de factoriels par la formule

$$\binom{n}{p} = \frac{n!}{p!(n-p)!}. \quad (1)$$

pour tout $n \in \mathbb{N}, p \in \mathbb{N}, p \leq n$.

Cependant ce n'est pas cette formule qui nous intéresse pour pouvoir générer les combinaisons, mais la formule du triangle de Pascal.

$$\begin{aligned} \binom{n}{p} &= \binom{n-1}{p-1} + \binom{n-1}{p} && \text{lorsque } p \neq 0 \\ &= 1 && \text{lorsque } p = 0 \end{aligned} \quad (2)$$

Preuve Tout d'abord, lorsque $p = 0$, nous avons un seul ensemble possible qui est l'ensemble vide. Supposons maintenant que $p \neq 0$. Posons $E_{n-1} = \{e_1, \dots, e_{n-1}\}$ et $E_n = \{e_1, \dots, e_n\} = E_{n-1} \cup \{e_n\}$. Nous allons construire un élément $F \in \mathcal{C}_p(E_n)$.

Nous avons deux cas possibles :

1. $e_n \in F$ et $F \setminus \{e_n\} \in \mathcal{C}_{p-1}^{p-1}$, soit $\binom{n-1}{p-1}$ possibilités.
2. $e_n \notin F$ et $F \in \mathcal{C}_{p-1}^{p-1}$, soit $\binom{n-1}{p}$ possibilités.

Donc au total nous avons $\binom{n-1}{p-1} + \binom{n-1}{p}$ possibilités.

Question 6 Utilisez cette preuve ensembliste de (??) pour définir un générateur `generateurCombinaisons(E,p)` énumérant les éléments de $\mathcal{C}_p(E)$.

Solution Le type de E doit définir les slices.

```
def generateurCombinaisons(E,p):
    if p==0:
        yield []
    else:
        for combinaison in generateurCombinaisons(E[1:],p-1):
            yield combinaison+[E[0]]
        if len(E) > p:
            yield from generateurCombinaisons(E[1:],p)
```

2.5 Énumération des mots sur un alphabet

Définition 2.6 (*Mots de longueur fixé*)

Soit $\mathcal{A}$ un alphabet (un ensemble fini de symboles) et $l \in \mathbb{N}$.

On notera $\mathcal{A}^l$ l'ensemble des mots sur $\mathcal{A}$ de longueur l .

Pour $l = 0$, $\mathcal{A}$ est toujours égal à $\{\varepsilon\}$, où ε est le mot vide qui est l'unique mot de longueur 0.

Question 7 Définissez un générateur `generateurMots(alphabet,l)` énumérant tous les éléments de alphabet^l , où `alphabet` sera un tuple ou une chaîne de caractères.

Indication Dans le langage python, le mot vide est "" et la concaténation de deux mots s'effectue avec l'opérateur +. En déduire un générateur `generateurMotsBinaires(l)` qui énumère tous les mots binaires de longueur l .

Solution

```
def generateurMots(alphabet,l): #l est la longueur des mots
    if l == 0:
        yield ""
    else:
        for mot in generateurMots(alphabet,l-1):
            for lettre in alphabet:
                yield mot+lettre
```

Définition 2.7 (*Langage $\mathcal{A}^*$*)

Le langage $\mathcal{A}^*$ est l'ensemble des mots sur l'alphabet $\mathcal{A}$. Il se définit de manière formelle par

$$\mathcal{A}^* = \bigcup_{l \in \mathbb{N}} \mathcal{A}^l.$$

Question 8 Définissez un générateur `generateurLangage(alphabet)` qui énumère tous les mots du langage $\mathcal{A}^*$.

Indication Nous devons choisir un ordre pour énumérer les mots de $\mathcal{A}^*$: on commence par énumérer le mot vide, puis les mots de longueur 1, puis ceux de longueur 2 et ainsi de suite.

Solution

```
def generateurLangage(alphabet):
    l = 0
    while True:
        for mot in generateurMots(alphabet,l):
            yield mot
        l +=1
```

3 Génération aléatoire sur un ensemble

Les itérateurs et les générateurs permettent d'énumérer tous les éléments d'un ensemble. Cela peut être utile afin de tester par exemple l'exécution d'un algorithme sur toutes les entrées possibles. Mais on peut également souhaiter récupérer un seul élément selon une distribution de probabilité. C'est ce que propose un générateur aléatoire qui va retourner un élément de E avec une distribution de probabilité fixée.

3.1 Génération aléatoire avec l'équiprobabilité en utilisant choice

La distribution de probabilité la plus usuelle est l'équiprobabilité où chaque élément de l'ensemble E possède la même probabilité d'être le résultat de l'expérience aléatoire.

Définition 3.1 (*Équiprobabilité*)

Soit E un ensemble fini et P une distribution de probabilité sur E . Nous avons l'équiprobabilité (ou encore la distribution uniforme) sur l'espace probabilisé (E, P) lorsque, pour tout $e \in E$,

$$\Pr(r = e) = \frac{1}{\text{card}(E)},$$

où r est le résultat de l'expérience aléatoire.

Dans le langage python, si l'ensemble est codé avec un objet itérable, le module `random` dispose d'une fonction `choice` pour retourner un élément avec l'équiprobabilité.

```
import random
print(random.choice([4,8,3,1]))
print(random.choice(range(10)))
print(random.choice("bonjour"))
```

Question 9 Écrivez une fonction `generationProduitCartesien(A,B)` qui retourne un élément de $A \times B$ avec l'équiprobabilité. Effectuez ensuite le tirage aléatoire avec l'équiprobabilité d'une carte d'un jeu de 32 cartes en utilisant cet fonction.

Solution

```
def generationProduitCartesien(A,B):
    return random.choice(A),random.choice(B)

#tirage d'une carte d'un jeu de 32 cartes
carte = generationProduitCartesien(hauteur, couleur)
```

Question 10 Vérifiez que la fonction `generationMot(alphabet,l)` suivante effectue le tirage aléatoire avec l'équiprobabilité d'un mot sur `alphabet` de longueur l . Cette fonction utilise la méthode `join` sur les chaînes de caractères qui permet de construire une chaîne de caractères en concaténant des chaînes de caractères contenues dans une liste.

Par exemple, si `L = ["a","b","c"]` alors `"".join(L)` renvoie `"abc"`.


```
def generationMot(alphabet,l):#l est la longueur des mots
    if l == 0:
        return ""
    return "".join([random.choice(alphabet) for i in range(l)])
```

3.2 Génération aléatoire avec l'équiprobabilité par des essais de Bernoulli

Définition 3.2 (*Loi de Bernoulli*)

Soit $E = \{0, 1\}$ (ou $\{\text{Vrai}, \text{Faux}\}$, ou encore $\{\text{Echec}, \text{Succès}\}$).

On définit une loi de Bernoulli de paramètre p , avec $p \in]0, 1[$ pour effectuer une expérience aléatoire de résultat $r \in E$ telle que

$$\begin{cases} \Pr(r = 1) &= p \\ \Pr(r = 0) &= 1 - p \end{cases}$$

On appelle essai de Bernoulli cette expérience aléatoire réalisée une fois.

Remarque 3.1 (*loi de Bernoulli et équiprobabilité*)

Nous avons l'équiprobabilité avec $p = 1/2$, car dans ce cas nous avons la même probabilité d'obtenir chacun des deux résultats.

Un essai de Bernoulli qui retourne vrai ou faux s'effectue avec le module **random** de la manière suivante :

```
def bernoulli(p):
    return random.random() < p
```

Proposition 3.1 (*Construction d'un élément avec l'équiprobabilité en utilisant des essais de Bernoulli*)

Soit E un ensemble fini. Supposons qu'il existe une méthode de construction avec des choix successifs tels que chaque choix offre deux possibilités qui permettent chacune d'obtenir par la suite autant d'éléments de E . En utilisant un essai de Bernoulli de paramètre $1/2$ pour chacun de ces choix, nous obtenons un élément de E avec l'équiprobabilité.

Exemple Construction d'un mot binaire de longueur l .

On effectue l choix :

Choix 1 : on choisit 0 ou 1 pour la première lettre.

Choix 2 : on choisit 0 ou 1 pour la seconde lettre.

⋮

Choix l : on choisit 0 ou 1 pour la dernière lettre.

Question 11 En reprenant la construction de la proposition ??, définissez une fonction `generationMotBinaire(E)` qui génère un mot binaire de longueur l avec l'équiprobabilité. Vous utiliserez la méthode `join` pour écrire cette fonction en une ligne.

Solution

```
def generationMotBinaire(l):
    return "".join(["0" if bernoulli(.5) else "1" for i in range(l)])
```

Question 12 En reprenant la construction de la proposition ??, définissez une fonction `generationSousEnsemble(E)` qui génère un sous-ensemble de E avec l'équiprobabilité, toujours avec une seule ligne d'instruction.

Solution

```
def generationSousEnsemble(E):  
    return [element for element in E if bernoulli(.5)]
```